Control Theory and Technology (2023) 21:72–88
https://doi.org/10.1007/s11768-023-00127-0

RESEARCH ARTICLE

Tuning and implementation variants of discrete-time ADRC

Gernot Herbst[1] · **Rafal Madonski**[2]

Received: 29 September 2022 / Revised: 14 November 2022 / Accepted: 23 November 2022 / Published online: 9 March 2023
© The Author(s), under exclusive licence to South China University of Technology and Academy of Mathematics and Systems Science, Chinese Academy of Sciences 2023

Abstract
Practical implementations of active disturbance rejection control (ADRC) will almost always take place in discretized form. Since applications may have quite different needs regarding their discrete-time controllers, this article summarizes and extends the available set of ADRC implementations to provide a suitable variant for as many as possible use cases. In doing so, the gap between quasi-continuous and discrete-time controller tuning is closed for applications with low sampling frequencies. The main contribution of this article is the derivation of three different discrete-time implementations of error-based ADRC. It is shown that these are almost one-to-one counterparts of existing output-based implementations, to the point where transfer functions and coefficients can be reused in unaltered form. In this way, error-based implementations become firmly rooted in the established landscape of discrete-time ADRC. Furthermore, it becomes possible to equip error-based variants with windup protection abilities known from output-based ADRC.

Keywords Active disturbance rejection control (ADRC) · Error-based form · Controller tuning · Discretization

1 Introduction

1.1 Background and related work

Active disturbance rejection control (ADRC) [1] is a highly practical approach for solving real-world control problems. As the dominance of PID controller families in this domain is undisputed despite decades of control systems research, the widespread interest ADRC has sparked among both scientists and engineers is all the more remarkable, since it has already found its way into numerous applications [2–5]. Some foundational milestones enabling this trajectory deep into the territory of PID control can be seen in the development of a linear version [6], an easy tuning rule [7], and its implementation in discrete-time form [8]. Later enhancements include additional features for industrial practice [9,10], methods for improved disturbance rejection [11], or the incorporation of known plant information [12,13].

An important barrier to be overcome for real-world adoption is not a technical one, but more a matter of understanding. With its origins in (nonlinear) state space, ADRC might appear unfamiliar to the practicing engineer. To cater to such needs, a multitude of studies provide insights and understanding of ADRC from a classical, frequency-domain control engineering perspective [14–17].

Taking this avenue also for the actual implementation—i.e., building ADRC using transfer functions—not only eases understanding, but also increases efficiency and allows to even reuse existing building blocks from classical control engineering. For discrete-time ADRC, this was put forward by [17], with an even more optimized variant in [18] that comes very close to the runtime performance of PID controllers. These are implementations we will build upon in this article.

A variant of ADRC, using the control error as estimated variable within the observer, was introduced in [19,20], and accordingly named *error-based ADRC*. When viewed in transfer function form, it becomes apparent that error-based

The work of R. Madonski was supported by the Fundamental Research Funds for the Central Universities (Project no. 21620335).

✉ Gernot Herbst
gernot.herbst@fh-zwickau.de

Rafal Madonski
rafal.madonski@jnu.edu.cn

[1] University of Applied Sciences Zwickau, Zwickau, Germany

[2] Energy and Electricity Research Center, Jinan University, Guangzhou, China

ADRC is a one-degree-of-freedom (1DOF) controller, as only one transfer function is needed—a feedback controller in a structure very well known from classical control engineering. Applications studied so far include quadrocopters [21], DC–DC converters [22], and thermal processes [23]. The firm connection between output- and error-based ADRC has been established in continuous-time domain [24]. To the best of our knowledge, however, the task of discretization of error-based ADRC has not yet been properly addressed, apart from an approximate solution using bilinear transform in [25]. We therefore want to extend this connection to the discrete-time domain in this article, providing ready-to-use variants that are exactly rooted in the discretization process of ADRC introduced by [8].

1.2 Contributions and structure of this article

This article aims at increasing the versatility of discrete-time ADRC in two aspects. Firstly, tuning rules for the outer state-feedback controller based on bandwidth parameterization [7] with discrete-time pole placement are introduced, which are needed to consistently achieve the desired bandwidth, even for applications with low sampling frequencies. At the same time, a bridge is built to the commonly used continuous-time design. Existing transfer function implementations of discrete-time ADRC from [17] and [18] are reviewed and retrofitted with these results. Secondly, discrete-time state space and transfer function implementations of error-based ADRC [25] are developed. These exhibit strong connections with classical (output-based) variants up to equivalent transfer function coefficients, and enhance error-based ADRC with flexible control signal limitation and windup protection.

The remainder of this article is structured as follows: Sect. 2 briefly recapitulates continuous-time ADRC, which we will need for deriving the discrete-time implementations. Building on that, Sect. 3 summarizes the current practice of implementing discrete-time ADRC in state space form. As will be shown, it relies on quasi-continuous tuning of the outer feedback controller within ADRC. Section 4 will therefore put the commonly used bandwidth parameterization approach of the controller on the basis of discrete-time pole placement. Two existing transfer function implementations of discrete-time ADRC are revisited in Sect. 5, now providing tuning equations based on the results of Section 4. For error-based ADRC, three different implementations are introduced in Sect. 6, all as direct counterparts of existing output-based variants. The article concludes after presenting an example in Sect. 7 that compares output- and error-based ADRC in a power electronics control application.

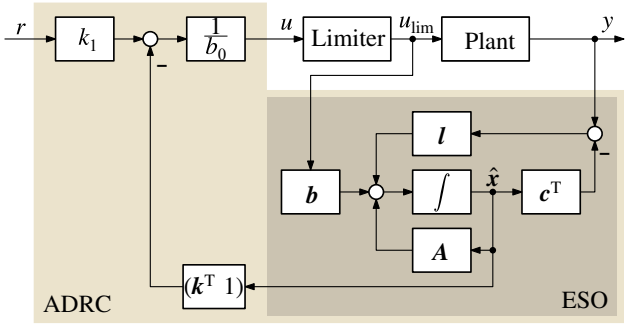


Fig. 1 Continuous-time ADRC in state space, with an external, user-defined control signal limiter

2 Summary of continuous-time ADRC

2.1 Structure and control law

All discrete-time implementations discussed in this article are based on the linear variant of ADRC introduced in [7], which we consider the most widely used form. We will briefly summarize it in this section. The control law of continuous-time linear ADRC in state space as shown in Fig. 1 reads:

$$u(t) = \frac{1}{b_0} \cdot \left(k_1 \cdot r(t) - \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \hat{\boldsymbol{x}}(t)\right) \tag{1}$$

with $\boldsymbol{k}^{\mathrm{T}} = \left(k_1 \cdots k_n\right)$ and $\hat{\boldsymbol{x}} = \left(\hat{x}_1 \cdots \hat{x}_{n+1}\right)^{\mathrm{T}}$.

In (1), the unity-gain factor in the state-feedback gain vector corresponds to the rejection of the total disturbance. The vector of states $\hat{\boldsymbol{x}}(t)$ is provided by a Luenberger observer. Its plant model most commonly consists of an integrator chain disturbed by a constant input disturbance (the total disturbance $f(t)$): $y^{(n)}(t) = b_0 \cdot u(t) + f(t)$. That way, only the plant order n and an estimated plant gain b_0 are required as far as modeling is concerned. The observer equations are therefore given as

$$\dot{\hat{\boldsymbol{x}}}(t) = \boldsymbol{A} \cdot \hat{\boldsymbol{x}}(t) + \boldsymbol{b} \cdot u_{\lim}(t) + \boldsymbol{l} \cdot \left(y(t) - \boldsymbol{c}^{\mathrm{T}} \cdot \hat{\boldsymbol{x}}(t)\right) \tag{2}$$

with

$$\boldsymbol{A} = \begin{pmatrix} \boldsymbol{0}^{n\times 1} & \boldsymbol{I}^{n\times n} \\ 0 & \boldsymbol{0}^{1\times n} \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} \boldsymbol{0}^{(n-1)\times 1} \\ b_0 \\ 0 \end{pmatrix},$$

$$\boldsymbol{l} = \left(l_1 \cdots l_{n+1}\right)^{\mathrm{T}}, \quad \boldsymbol{c}^{\mathrm{T}} = \left(1\ \boldsymbol{0}^{1\times n}\right).$$

Note that in (2), the observer uses the limited control signal $u_{\lim}(t)$ instead of $u(t)$, which is a very simple, but effective method of avoiding integrator windup [10,26]—a very important selling point for using ADRC in practice. That way, no further anti-windup measures have to be implemented, let

alone tuned. Finally, it might also be possible that the control signal limitation takes place externally, beyond the user's influence. For these cases, $u_{\lim}(t)$ provides a feedback path for a measured value of the actual (limited) control signal.

2.2 Controller tuning

We will now briefly discuss tuning the gain vector $\boldsymbol{k}^{\mathrm{T}}$ of the outer state-feedback controller from (1). This will also be the basis for developing discrete-time tuning rules in Sect. 4.1. The $1/b_0$ gain block in Fig. 1 normalizes the plant gain, which, together with feeding back the estimated total disturbance, approximately creates an undisturbed "virtual plant" with unity-gain integrator chain dynamics $y^{(n)}(t) = u_0(t)$ to be controlled by the controller $u_0(t) = -\boldsymbol{k}^{\mathrm{T}} \cdot \left(\hat{x}_1 \cdots \hat{x}_n\right)^{\mathrm{T}}$. A state space representation of this virtual plant can be given as

$$\begin{cases} \dot{\boldsymbol{x}}_{\mathrm{VP}}(t) = \boldsymbol{A}_{\mathrm{VP}} \cdot \boldsymbol{x}_{\mathrm{VP}}(t) + \boldsymbol{b}_{\mathrm{VP}} \cdot u_0(t), \\ y(t) = \boldsymbol{c}_{\mathrm{VP}}^{\mathrm{T}} \cdot \boldsymbol{x}_{\mathrm{VP}}(t) \end{cases} \tag{3}$$

with

$$\boldsymbol{A}_{\mathrm{VP}} = \begin{pmatrix} \boldsymbol{0}^{(n-1)\times 1} & \boldsymbol{I}^{(n-1)\times(n-1)} \\ 0 & \boldsymbol{0}^{1\times(n-1)} \end{pmatrix},$$
$$\boldsymbol{b}_{\mathrm{VP}} = \begin{pmatrix} \boldsymbol{0}^{(n-1)\times 1} \\ 1 \end{pmatrix}, \quad \boldsymbol{c}_{\mathrm{VP}}^{\mathrm{T}} = \left(1\ \boldsymbol{0}^{1\times(n-1)}\right).$$

The most popular tuning method for linear ADRC was introduced in [7] as *bandwidth parameterization*. Its simplicity comes from a single tuning parameter, namely, the desired bandwidth ω_{CL}. In this approach, all poles of the closed loop are placed at a common location $-\omega_{\mathrm{CL}}$:

$$\begin{aligned} (\lambda + \omega_{\mathrm{CL}})^n &\stackrel{!}{=} \det\left(\lambda \boldsymbol{I} - \left(\boldsymbol{A}_{\mathrm{VP}} - \boldsymbol{b}_{\mathrm{VP}}\boldsymbol{k}^{\mathrm{T}}\right)\right) \\ &= \lambda^n + k_n\lambda^{n-1} + \ldots + k_2\lambda + k_1. \end{aligned} \tag{4}$$

After binomal expansion, the controller gains k_i can be obtained from (4):

$$k_i = \frac{n!}{(n-i+1)! \cdot (i-1)!} \cdot \omega_{\mathrm{CL}}^{n-i+1}, \quad \forall i = 1, \ldots, n. \tag{5}$$

Equation (5) yields the very simple controller tuning equations $k_1 = \omega_{\mathrm{CL}}$ (for first-order ADRC), and $k_1 = \omega_{\mathrm{CL}}^2$, $k_2 = 2\omega_{\mathrm{CL}}$ (for second-order ADRC). We will come back to these in Sect. 4.2.

2.3 Observer tuning

In a similar fashion, tuning the observer can be done by placing the poles of the observer system matrix $\left(\boldsymbol{A} - \boldsymbol{l}\boldsymbol{c}^{\mathrm{T}}\right)$. Using bandwidth parameterization once more, an (obviously higher) bandwidth has to be chosen for the observer, which we will—following [27]—express as $k_{\mathrm{ESO}} \cdot \omega_{\mathrm{CL}}$. The equation for pole placement then reads:

$$\begin{aligned} (\lambda + k_{\mathrm{ESO}}\omega_{\mathrm{CL}})^{n+1} &\stackrel{!}{=} \det\left(\lambda \boldsymbol{I} - \left(\boldsymbol{A} - \boldsymbol{l}\boldsymbol{c}^{\mathrm{T}}\right)\right) \\ &= \lambda^{n+1} + l_1\lambda^n + \ldots + l_n\lambda + l_{n+1}. \end{aligned} \tag{6}$$

The observer gains can be obtained from (6) after binominal expansion as follows:

$$l_i = \frac{(n+1)!}{(n-i+1)! \cdot i!} \cdot \left(k_{\mathrm{ESO}} \cdot \omega_{\mathrm{CL}}\right)^i, \quad \forall i = 1, \ldots, n+1. \tag{7}$$

3 Discrete-time ADRC

3.1 Structure and control law

As the control law (1) of linear ADRC consists of static state feedback, its discrete-time counterpart is trivially obtained as

$$u(k) = \frac{1}{b_0} \cdot \left(k_1 \cdot r(k) - \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \hat{\boldsymbol{x}}(k)\right) \tag{8}$$

with $\boldsymbol{k}^{\mathrm{T}} = \left(k_1 \cdots k_n\right)$.

When developing the discrete-time version of the extended state observer, the so-called *current observer* approach is being used to let the current estimate $\hat{\boldsymbol{x}}(k)$ be based on the most recent measurement $y(k)$ [8,28]. For a more detailed derivation, we refer the reader to [27], and only present the resulting equations for the discrete-time observer dynamics here:

$$\hat{\boldsymbol{x}}(k) = \boldsymbol{A}_{\mathrm{ESO}} \cdot \hat{\boldsymbol{x}}(k-1) + \boldsymbol{b}_{\mathrm{ESO}} \cdot u_{\lim}(k-1) + \boldsymbol{l} \cdot y(k) \tag{9}$$

with $\boldsymbol{l} = \left(l_1 \cdots l_{n+1}\right)^{\mathrm{T}}$, $\boldsymbol{A}_{\mathrm{ESO}} = \boldsymbol{A}_{\mathrm{d}} - \boldsymbol{l} \cdot \boldsymbol{c}_{\mathrm{d}}^{\mathrm{T}} \cdot \boldsymbol{A}_{\mathrm{d}}$, and $\boldsymbol{b}_{\mathrm{ESO}} = \boldsymbol{b}_{\mathrm{d}} - \boldsymbol{l} \cdot \boldsymbol{c}_{\mathrm{d}}^{\mathrm{T}} \cdot \boldsymbol{b}_{\mathrm{d}}$.

In (9), $\boldsymbol{A}_{\mathrm{d}}$, $\boldsymbol{b}_{\mathrm{d}}$, and $\boldsymbol{c}_{\mathrm{d}}^{\mathrm{T}}$ are obtained from zero-order hold (ZOH) discretization with a sampling interval T of their respective counterparts in (2), as recommended by [8]:

$$\boldsymbol{A}_{\mathrm{d}} = \boldsymbol{I} + \sum_{i=1}^{\infty} \frac{\boldsymbol{A}^i T^i}{i!}, \quad \boldsymbol{b}_{\mathrm{d}} = \left(\sum_{i=1}^{\infty} \frac{\boldsymbol{A}^{i-1} T^i}{i!}\right) \cdot \boldsymbol{b}, \quad \boldsymbol{c}_{\mathrm{d}}^{\mathrm{T}} = \boldsymbol{c}^{\mathrm{T}}.$$

Controller (8) and observer (9) are shown as a block diagram in Fig. 2. Once again, one can recognize ADRC's signature ingredients: extended state observer (built around a simple plant model with input disturbance), disturbance

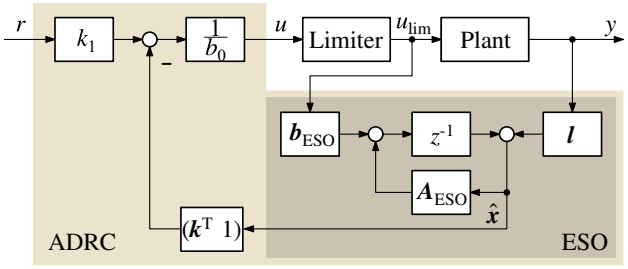


Fig. 2 Control loop with discrete-time ADRC, state space implementation

rejection (unity-gain feedback of the state $\hat{x}_{n+1}$), and plant gain inversion (multiplication by $1/b_0$ to obtain $u(k)$).

3.2 Controller tuning

The outer state feedback controller of ADRC consists of a static feedback gain vector $\boldsymbol{k}^{\text{T}}$, which means that there are no dynamics to be discretized for the controller. Assuming that the sampling interval T is negligible compared to the dynamics of the closed control loop, the continuous-time design from Sect. 2.2 can be employed with very good accuracy for a discrete-time implementation, as well.

Indeed, to the best of the authors' knowledge, this quasi-continuous controller design within a discrete-time linear ADRC implementation has, up to now, always been performed this way; see [29–34] for some examples. For applications with relatively large sampling intervals, however, the quasi-continuous design does not exactly lead to the desired closed-loop behavior. To round off ADRC controller tuning even for these edge cases, we therefore want to introduce discrete-time tuning rules for $\boldsymbol{k}^{\text{T}}$ in this article, which will be presented in Sect. 4.

3.3 Observer tuning

Looking back at (9), it becomes obvious that the abbreviation A_{ESO} is the system matrix of the discrete-time observer. Corresponding to the continuous-time case, we can use the pole placement approach of bandwidth parameterization for tuning the observer. Before doing so, we need to map the desired common pole from its s-plane to the z-domain location z_{ESO}:

$$\det\left(z\boldsymbol{I}-\boldsymbol{A}_{\text{ESO}}\right) \stackrel{!}{=} (z-z_{\text{ESO}})^{n+1} \tag{10}$$

with $z_{\text{ESO}} = \mathrm{e}^{-k_{\text{ESO}}\cdot\omega_{\text{CL}}\cdot T}$.

As shown before, for example in [8,27], this results in the observer gains $l_1 = 1 - z_{\text{ESO}}^2$ and $l_2 = \frac{1}{T}\cdot(1-z_{\text{ESO}})^2$ for the first-order case, and three gain values $l_1 = 1 - z_{\text{ESO}}^3$, $l_2 = \frac{3}{2T}\cdot(1-z_{\text{ESO}})^2\cdot(1+z_{\text{ESO}})$, $l_3 = \frac{1}{T^2}\cdot(1-z_{\text{ESO}})^3$ for second-order ADRC.

4 Discrete-time controller tuning

4.1 Bandwidth parameterization

In a discrete-time implementation of ADRC, all inputs and outputs are updated only once per sampling interval T. The observer will, naturally, provide estimates for the outer state-feedback controller and the total disturbance only at these time instants, as well. A discrete-time design of the controller $\boldsymbol{k}^{\text{T}}$ therefore requires a discrete-time model of the virtual plant (3), as well. Assuming a zero-order hold behavior of the actuator, we can derive the following state-space model:

$$\begin{cases} \hat{\boldsymbol{x}}_{\text{VP}}(k+1) = \boldsymbol{A}_{\text{VP,d}}\cdot\hat{\boldsymbol{x}}_{\text{VP}}(k) + \boldsymbol{b}_{\text{VP,d}}\cdot u(k), \\ y_{\text{VP}}(k) = \boldsymbol{c}_{\text{VP,d}}^{\text{T}}\cdot\hat{\boldsymbol{x}}_{\text{VP}}(k) \end{cases} \tag{11}$$

with

$$\boldsymbol{A}_{\text{VP,d}} = \boldsymbol{I} + \sum_{i=1}^{\infty}\frac{\boldsymbol{A}_{\text{VP}}^i T^i}{i!},$$

$$\boldsymbol{b}_{\text{VP,d}} = \left(\sum_{i=1}^{\infty}\frac{\boldsymbol{A}_{\text{VP}}^{i-1} T^i}{i!}\right)\cdot\boldsymbol{b}_{\text{VP}},\quad \boldsymbol{c}_{\text{VP,d}}^{\text{T}} = \boldsymbol{c}_{\text{VP}}^{\text{T}}.$$

Similar to the observer tuning procedure described in Sect. 3.3, we can now apply pole placement with bandwidth parameterization for the discrete-time controller. Firstly, we map the desired location for all closed-loop poles from s- to z-domain:

$$z_{\text{CL}} = \mathrm{e}^{-\omega_{\text{CL}}T}, \tag{12}$$

and then the design equation will read:

$$\det\left(z\boldsymbol{I}-\left(\boldsymbol{A}_{\text{VP,d}}-\boldsymbol{b}_{\text{VP,d}}\boldsymbol{k}^{\text{T}}\right)\right) \stackrel{!}{=} (z-z_{\text{CL}})^n. \tag{13}$$

The most relevant cases for a practical implementation are $n=1$ and $n=2$, where ADRC will be used to replace PI and PID controllers, respectively. For the first-order case, $\boldsymbol{A}_{\text{VP,d}}$ and $\boldsymbol{b}_{\text{VP,d}}$ are only scalars:

$$\boldsymbol{A}_{\text{VP,d}} = \left(1\right)\ \text{ and }\ \boldsymbol{b}_{\text{VP,d}} = \left(T\right).$$

This means that (13) will be reduced to the following simple equation:

$$z + (Tk_1 - 1) \stackrel{!}{=} z - z_{\text{CL}},$$

from which we can easily obtain the discrete-time controller tuning equation for k_1 as

$$k_1 = \frac{1-z_{\text{CL}}}{T}. \tag{14}$$

In the second-order case, $\boldsymbol{A}_{\mathrm{VP,d}}$ and $\boldsymbol{b}_{\mathrm{VP,d}}$ are

$$\boldsymbol{A}_{\mathrm{VP,d}} = \begin{pmatrix} 1 & T \\ 0 & 1 \end{pmatrix} \text{ and } \boldsymbol{b}_{\mathrm{VP,d}} = \begin{pmatrix} \frac{1}{2}T^2 \\ T \end{pmatrix}.$$

The design Eq. (13) now becomes

$$z^2 + \left(\frac{k_1 T^2}{2} + k_2 T - 2\right) \cdot z + \left(\frac{k_1 T^2}{2} - k_2 T + 1\right) \stackrel{!}{=} z^2 - 2z_{\mathrm{CL}} \cdot z + z_{\mathrm{CL}}^2,$$

which we can solve for the controller gains k_1 and k_2, obtaining

$$k_1 = \frac{(1 - z_{\mathrm{CL}})^2}{T^2} \text{ and } k_2 = \frac{4 - (1 + z_{\mathrm{CL}})^2}{2T}. \tag{15}$$

4.2 Approximation for high sampling frequencies

For the common case of sufficiently high sampling frequencies, we now want to provide a connection between discrete- and continuous-time controller tuning for ADRC, specifically for the practically relevant cases $n = 1$ and $n = 2$.

Let us revisit (12), remembering the power series characterization of a real-valued exponential function. This allows to express z_{CL} as

$$z_{\mathrm{CL}} = \mathrm{e}^{-\omega_{\mathrm{CL}} T} = 1 + (-\omega_{\mathrm{CL}} T) + \frac{(-\omega_{\mathrm{CL}} T)^2}{2!} + \frac{(-\omega_{\mathrm{CL}} T)^3}{3!} + \ldots,$$

which we can truncate after the second term if the sampling interval T is small enough to warrant $\omega_{\mathrm{CL}} T \ll 1$. Then we obtain the following approximation for z_{CL}:

$$z_{\mathrm{CL}} \approx 1 - \omega_{\mathrm{CL}} T. \tag{16}$$

For the first-order case, putting (16) in (14) gives

$$k_1 = \frac{1 - z_{\mathrm{CL}}}{T} \approx \frac{1 - (1 - \omega_{\mathrm{CL}} T)}{T} = \omega_{\mathrm{CL}}, \tag{17}$$

which corresponds to the continuous-time tuning value obtained by (5).

In the same way, we obtain the following approximations for second-order ADRC:

$$\begin{aligned} k_1 &= \frac{(1 - z_{\mathrm{CL}})^2}{T^2} \approx \frac{(1 - (1 - \omega_{\mathrm{CL}} T))^2}{T^2} = \omega_{\mathrm{CL}}^2, \\ k_2 &= \frac{4 - (1 + z_{\mathrm{CL}})^2}{2T} \approx \frac{4 - (1 + (1 - \omega_{\mathrm{CL}} T))^2}{2T} \\ &= \omega_{\mathrm{CL}} \cdot \left(2 - \frac{\omega_{\mathrm{CL}} T}{2}\right) \approx 2\omega_{\mathrm{CL}}. \end{aligned} \tag{18}$$

We can now summarize the first contributions of this article:

- It can be confirmed that the established quasi-continuous controller tuning can indeed be applied to discrete-time ADRC provided that the condition $\omega_{\mathrm{CL}} T \ll 1$ holds.
- The bandwidth parameterization tuning rules for the discrete-time controller derived in Sect. 4.1 are a "backward compatible" generalization of the existing practice, delivering correct results regardless of the choice of the sampling interval T.

4.3 Comparison of continuous- and discrete-time tuning

In Sect. 3.2, we mentioned that the established way of controller tuning within discrete-time ADRC is to simply use the continuous-time described in Sect. 2.2. The above-mentioned examples [29–34] obviously demonstrate as well that no problems are to be expected in most practical settings. Therefore, when will the dedicated discrete-time tuning just introduced in Sect. 4.1 make a difference?

Obviously the sampling interval T must be small enough compared to the dynamics of the closed loop. As already evident from Sect. 4.2, we will therefore let our analysis be based on the relative factor $\omega_{\mathrm{CL}} T$. To give a visual impression of the difference between continuous-time controller tuning as given by (4) and a discrete-time design obtained through (13), the relative factor of continuous-time (CT) vs. discrete-time (DT) controller gains are shown in Fig. 3 for first- and second-order ADRC.

For $n = 1$ (shown in Fig. 3a), the relative factor is

$$\frac{k_1^{\mathrm{CT}}(\omega_{\mathrm{CL}} T)}{k_1^{\mathrm{DT}}(\omega_{\mathrm{CL}} T)} = \frac{\omega_{\mathrm{CL}}}{\frac{1 - z_{\mathrm{CL}}}{T}} = \frac{\omega_{\mathrm{CL}} T}{1 - \mathrm{e}^{-\omega_{\mathrm{CL}} T}}, \tag{19}$$

while for $n = 2$ (shown in Fig. 3b), the following two relations are obtained:

$$\begin{aligned} \frac{k_1^{\mathrm{CT}}(\omega_{\mathrm{CL}} T)}{k_1^{\mathrm{DT}}(\omega_{\mathrm{CL}} T)} &= \frac{\omega_{\mathrm{CL}}^2}{\frac{(1 - z_{\mathrm{CL}})^2}{T^2}} = \frac{(\omega_{\mathrm{CL}} T)^2}{\left(1 - \mathrm{e}^{-\omega_{\mathrm{CL}} T}\right)^2}, \\ \frac{k_2^{\mathrm{CT}}(\omega_{\mathrm{CL}} T)}{k_2^{\mathrm{DT}}(\omega_{\mathrm{CL}} T)} &= \frac{2\omega_{\mathrm{CL}}}{\frac{4 - (1 + z_{\mathrm{CL}})^2}{2T}} = \frac{4\omega_{\mathrm{CL}} T}{4 - \left(1 + \mathrm{e}^{-\omega_{\mathrm{CL}} T}\right)^2}. \end{aligned} \tag{20}$$

From the visualization of (19) and (20) in Fig. 3, it becomes obvious that using a continuous-time design in a discrete-time ADRC implementation may lead to serious problems above $\omega_{\mathrm{CL}} T \gtrapprox 0.1$. The controller gains obtained from continuous-time design are then overestimating the actually needed values, leading to a higher bandwidth than designed—and being closer to possible instability of the loop.

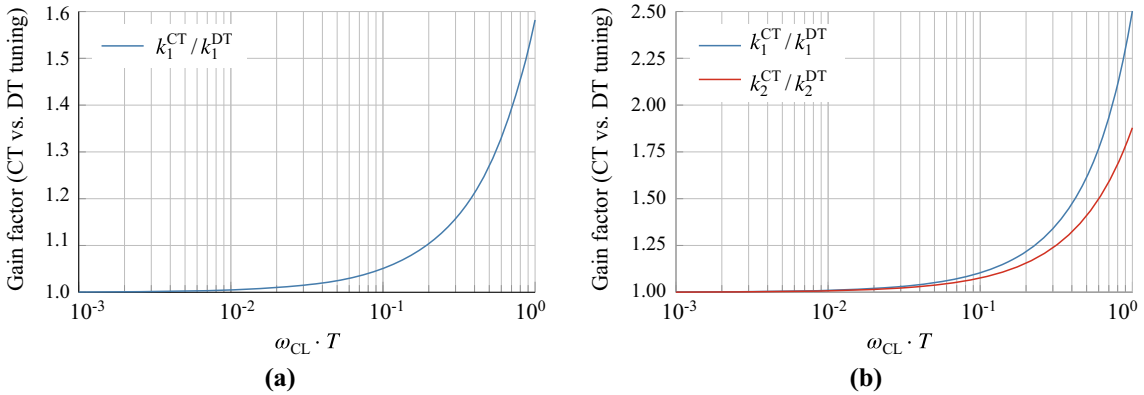


Fig. 3 Relation of controller gains obtained by continuous-time design (4) vs. discrete-time design (13) depending on the relative sampling interval $\omega_{CL}T$. As a practical rule of thumb, it can be said that below $\omega_{CL}T \lessapprox 0.05$ both continuous- and discrete-time design become indistinguishable. **a** First-order ADRC. **b** Second-order ADRC

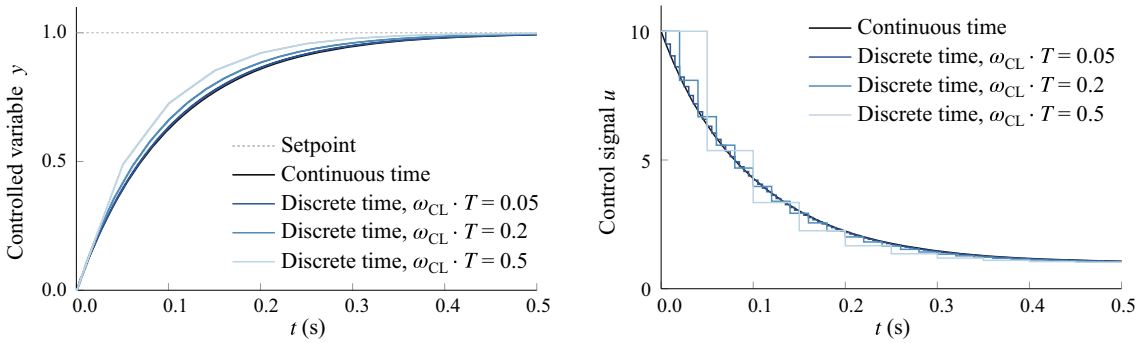


Fig. 4 Comparison of closed-loop step response and control signal for a control loop with a normalized, continuous-time plant $P(s) = \frac{1}{s+1}$ and first-order ADRC implementations tuned with $b_0 = 1$, $\omega_{CL} = 10$, and $k_{ESO} = 10$. For the discrete-time implementations with a "traditional" quasi-continuous controller design described in Sect. 2.2, the actual bandwidth deviates from the true continuous-time case with increasing sampling intervals—the controller gains are too high. These simulations are idealized (no measurement noise, zero delay between measurement of y and update of u) to focus attention on the effects of increased sampling intervals

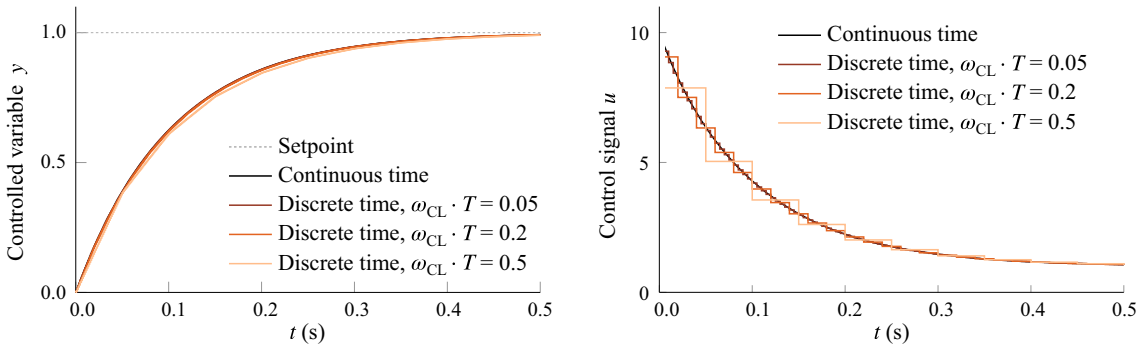


Fig. 5 Comparison of Fig. 4 repeated with the discrete-time controller design introduced in Sect. 4.1, instead of the quasi-continuous design previously employed for discrete-time ADRC. Continuous- and discrete-time implementations now always exhibit similar closed-loop dynamics

Conversely, a simple rule of thumb can be learned from Fig. 3. For relative sampling intervals below $\omega_{CL}T \lessapprox 0.05$, the closed-loop behaviors of a continuous- vs. discrete-time design will hardly be discernible.

Both findings are confirmed in a simple time-domain comparison of control loops with different discrete-time ADRC implementations using quasi-continuous and actual discrete-time controller designs in Fig. 4 and Fig. 5.

As a final side note it shall be mentioned that for $\omega_{\mathrm{CL}} T \gtrapprox 0.1$ there will, of course, be other problems to tackle in the control loop besides overestimated controller gains. With such large sampling intervals, any delay unaccounted for will quickly lead to oscillations or even instability.

5 Discrete-time transfer function implementations

In this section, we reiterate the derivation of two discrete-time transfer function implementations of ADRC introduced in [17] and [18], with slight variations that lead to easier understanding and simpler tuning equations. In contrast to previous work, however, we now give detailed coefficients for all transfer functions based on the discrete-time controller tuning rules developed in Sect. 4.1. This will also form the basis of introducing new discrete-time implementations of error-based ADRC in Sect. 6.

5.1 Transfer function implementation

The first step toward a discrete-time transfer function representation of the state-space implementation is made by z-transform of both controller (8) and observer (9):

$$u(z) = \frac{1}{b_0} \cdot \left(k_1 \cdot r(z) - \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \hat{\boldsymbol{x}}(z)\right), \tag{21}$$

$$\hat{\boldsymbol{x}}(z) = z^{-1} \cdot \boldsymbol{A}_{\mathrm{ESO}} \cdot \hat{\boldsymbol{x}}(z) + z^{-1} \cdot \boldsymbol{b}_{\mathrm{ESO}} \cdot u_{\mathrm{lim}}(z) + \boldsymbol{l} \cdot y(z). \tag{22}$$

Putting (21) in (22) leads to the closed-loop dynamics of the discrete-time observer. To be able to do that, we sacrifice the separate feedback path of the limited control signal u_{lim} back to the observer, as this feedback loop will now be "convoluted" in the closed-loop observer dynamics. With $u(z) = u_{\mathrm{lim}}(z)$ one obtains:

$$\hat{\boldsymbol{x}}(z) = \boldsymbol{\Phi}_{\mathrm{ESO}} \cdot \left(z^{-1} \cdot \frac{k_1}{b_0} \cdot \boldsymbol{b}_{\mathrm{ESO}} \cdot r(z) + \boldsymbol{l} \cdot y(z)\right). \tag{23}$$

In (23), we introduced the matrix $\boldsymbol{\Phi}_{\mathrm{ESO}}$ as a shorthand notation:

$$\boldsymbol{\Phi}_{\mathrm{ESO}} = \left(\boldsymbol{I} - z^{-1} \cdot \left(\boldsymbol{A}_{\mathrm{ESO}} - \frac{1}{b_0} \cdot \boldsymbol{b}_{\mathrm{ESO}} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right)\right)\right)^{-1}. \tag{24}$$

Transfer functions will relate the reference signal and the measured plant output to the updated controller output. The vector of estimated state variables $\hat{\boldsymbol{x}}(z)$ therefore needs to be eliminated. That can be achieved by putting (23) back into (21). The control law can now be written as

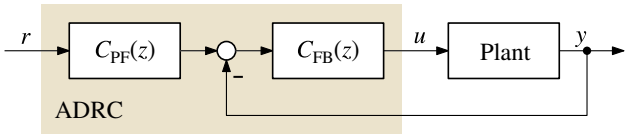


Fig. 6 Discrete-time transfer function implementation of ADRC

$$u(z) = \frac{k_1}{b_0} \cdot \left(1 - z^{-1} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \boldsymbol{\Phi}_{\mathrm{ESO}} \cdot \frac{1}{b_0} \cdot \boldsymbol{b}_{\mathrm{ESO}}\right) \cdot r(z) - \left(\frac{1}{b_0} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \boldsymbol{\Phi}_{\mathrm{ESO}} \cdot \boldsymbol{l}\right) \cdot y(z). \tag{25}$$

Equation (25) obviously represents a two-degrees-of-freedom control structure, albeit not a typical (or desirable) one. A discrete-time controller structure often encountered in practice is shown in Fig. 6 and consists of a feedback controller $C_{\mathrm{FB}}(z)$ and a prefilter $C_{\mathrm{PF}}(z)$ as follows:

$$u(z) = C_{\mathrm{FB}}(z) \cdot \left[C_{\mathrm{PF}}(z) \cdot r(z) - y(z)\right]. \tag{26}$$

This structure is also shown in Fig. 6. We can transform (25) into that form by factoring out the feedback transfer function. Now we can read the transfer functions of both feedback controller $C_{\mathrm{FB}}(z)$ and prefilter $C_{\mathrm{PF}}(z)$ as follows:

$$C_{\mathrm{FB}}(z) = \frac{1}{b_0} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \boldsymbol{\Phi}_{\mathrm{ESO}} \cdot \boldsymbol{l}, \tag{27}$$

$$C_{\mathrm{PF}}(z) = \frac{\frac{k_1}{b_0} \cdot \left(1 - z^{-1} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \boldsymbol{\Phi}_{\mathrm{ESO}} \cdot \frac{1}{b_0} \cdot \boldsymbol{b}_{\mathrm{ESO}}\right)}{\frac{1}{b_0} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \boldsymbol{\Phi}_{\mathrm{ESO}} \cdot \boldsymbol{l}}. \tag{28}$$

As its counterpart in the continuous time domain, the discrete-time feedback controller transfer function $C_{\mathrm{FB}}(z)$ includes an integrator—otherwise the control loop would not be able to reject constant disturbances and follow constant reference signal values with zero steady-state error, obviously. In [17] it was put forward to factor this pole at $z = 1$ out of $C_{\mathrm{FB}}(z)$:

$$C_{\mathrm{FB}}(z) = \Delta C_{\mathrm{FB}}(z) \cdot \frac{1}{1 - z^{-1}}.$$

Implementing $C_{\mathrm{FB}}(z)$ as a series connection of an accumulator and a discrete-time filter comes with the following advantages:

- One dependent coefficient of the denominator polynomial is removed, which means there is one coefficient less to compute and to store in an implementation. This also saves one multiplication at runtime.
- The $\Delta C_{\mathrm{FB}}(z)$ part of the feedback controller now exhibits the *incremental* [35] or *velocity* [36] form of a digital controller. Implementing the accumulator with clamping

therefore provides an easy but quite effective protection against integrator windup.

This is also the reason why (25) had to be brought into the form (26)—otherwise there would be a second integrator, namely in the transfer function from r to u. For the overall behavior, these two structures are equivalent in a mathematical sense, but having two integrators (one between r and u and one between y and u) would result in two integrator states running away to infinity for any nonzero reference r.

Putting everything together and applying discrete-time observer and controller tuning as described in Sect. 3.3 and Sect. 4.1, one obtains the following two transfer functions $C_{\mathrm{FB}}(z)$ and $C_{\mathrm{PF}}(z)$:

$$C_{\mathrm{FB}}(z) = \frac{\sum_{i=0}^{n} \beta_i z^{-i}}{1 + \sum_{i=1}^{n} \alpha_i z^{-i}} \cdot \frac{1}{1 - z^{-1}}, \tag{29}$$

$$C_{\mathrm{PF}}(z) = \frac{\frac{1}{\beta_0} \sum_{i=0}^{n+1} \gamma_i z^{-i}}{1 + \frac{1}{\beta_0} \sum_{i=1}^{n} \beta_i z^{-i}}. \tag{30}$$

For the first- and second-order case of ADRC, which are particularly relevant in practice, we provide a detailed set of equations for the α, β, γ coefficients based on the continuous-time tuning parameters ω_{CL} (desired closed-loop bandwidth) and k_{ESO} (relative observer bandwidth factor) in Tables 1 and 2, respectively. These results are—in the sense of Sect. 4.3—a more accurate generalization of the coefficients previously reported in [17], which used a quasi-continuous design for the controller gains. In (30) we factored out $\frac{1}{\beta_0}$, which allows to reuse the β coefficients for $C_{\mathrm{PF}}(z)$ and give more compact tuning equations for the γ coefficients.

Note that this discrete-time transfer function implementation does not fully replicate the behavior of its state-space ancestor. On the other hand, transfer functions can be implemented more efficiently, as their number of coefficients grows only linearly with the order of the controller n, as opposed to quadratic growth in the state-space case. The price for obtaining a very familiar structure with prefilter and feedback controller was to eliminate the separate feedback path of the limited controller output u_{lim}. This results in loss of the "out-of-the-box" windup protection coming with ADRC—a loss than can be partially compensated by implementation of a clamped integrator as part of $C_{\mathrm{FB}}(z)$. Luckily, it is not always an either/or situation. In the second transfer function approach, that we will introduce in Sect. 5.2, state-space features will be combined with transfer function efficiency.

Table 1 Discrete-time transfer function implementation of first-order ADRC: coefficients of the transfer functions $C_{\mathrm{FB}}(z)$ and $C_{\mathrm{PF}}(z)$ from (29) and (30), obtained by discrete-time pole placement and bandwidth parameterization with $z_{\mathrm{CL}} = \mathrm{e}^{-\omega_{\mathrm{CL}} T}$ (closed loop) and $z_{\mathrm{ESO}} = \mathrm{e}^{-k_{\mathrm{ESO}} \omega_{\mathrm{CL}} T}$ (observer)

Coefficient	Value for first-order ADRC
α_1	$-z_{\mathrm{CL}} z_{\mathrm{ESO}}^2$
β_0	$\frac{1}{b_0 T} \cdot \left[z_{\mathrm{CL}} z_{\mathrm{ESO}}^2 - 2 z_{\mathrm{ESO}} - z_{\mathrm{CL}} + 2 \right]$
β_1	$\frac{1}{b_0 T} \cdot \left[2 z_{\mathrm{CL}} z_{\mathrm{ESO}} - 2 z_{\mathrm{CL}} z_{\mathrm{ESO}}^2 + z_{\mathrm{ESO}}^2 - 1 \right]$
γ_0	$\frac{1 - z_{\mathrm{CL}}}{b_0 T}$
γ_1	$\frac{-2 z_{\mathrm{ESO}} \cdot (1 - z_{\mathrm{CL}})}{b_0 T}$
γ_2	$\frac{z_{\mathrm{ESO}}^2 \cdot (1 - z_{\mathrm{CL}})}{b_0 T}$

5.2 Dual-feedback transfer function implementation

A different transfer function implementation that preserves the feedback path of the limited control signal value $u_{\mathrm{lim}}(z)$ was introduced in [18]. The starting point of Sect. 5.1 is being used here, as well: the control law (21) and observer (22) in z-domain. The latter is being solved for $\hat{\boldsymbol{x}}(z)$:

$$\hat{\boldsymbol{x}}(z) = \left(\boldsymbol{I} - z^{-1} \boldsymbol{A}_{\mathrm{ESO}} \right)^{-1} \cdot \left(z^{-1} \cdot \boldsymbol{b}_{\mathrm{ESO}} \cdot u_{\mathrm{lim}}(z) + \boldsymbol{l} \cdot y(z) \right).$$

Putting this equation back in (21) yields a control law depicted in Fig. 7. It features three inputs r, y, and u_{lim}, as in the state-space case:

$$\begin{aligned} u(z) &= \frac{1}{b_0} \cdot \Big(k_1 \cdot r(z) - \begin{pmatrix} \boldsymbol{k}^{\mathrm{T}} & 1 \end{pmatrix} \cdot \left(\boldsymbol{I} - z^{-1} \boldsymbol{A}_{\mathrm{ESO}} \right)^{-1} \\ &\quad \cdot \left(z^{-1} \cdot \boldsymbol{b}_{\mathrm{ESO}} \cdot u_{\mathrm{lim}}(z) + \boldsymbol{l} \cdot y(z) \right) \Big) \\ &= \frac{k_1}{b_0} \cdot r(z) - C_{\mathrm{FBy}}(z) \cdot y(z) + C_{\mathrm{FBu}}(z) \cdot u_{\mathrm{lim}}(z). \end{aligned} \tag{31}$$

The two feedback transfer functions $C_{\mathrm{FBy}}(z)$ and $C_{\mathrm{FBu}}(z)$ in (31) share the same denominator and can be expressed as given in (32) and (33).

$$\begin{aligned} C_{\mathrm{FBy}}(z) &= \frac{1}{b_0} \cdot \begin{pmatrix} \boldsymbol{k}^{\mathrm{T}} & 1 \end{pmatrix} \cdot \left(\boldsymbol{I} - z^{-1} \boldsymbol{A}_{\mathrm{ESO}} \right)^{-1} \cdot \boldsymbol{l} \\ &= \frac{\sum_{i=0}^{n} \beta_i z^{-i}}{1 + \sum_{i=1}^{n+1} \alpha_i z^{-i}}, \end{aligned} \tag{32}$$

Table 2 Discrete-time transfer function implementation of second-order ADRC: coefficients of the transfer functions $C_{\mathrm{FB}}(z)$ and $C_{\mathrm{PF}}(z)$ from (29) and (30), obtained by discrete-time pole placement and bandwidth parameterization with $z_{\mathrm{CL}} = \mathrm{e}^{-\omega_{\mathrm{CL}} T}$ (closed loop) and $z_{\mathrm{ESO}} = \mathrm{e}^{-k_{\mathrm{ESO}} \omega_{\mathrm{CL}} T}$ (observer)

Coefficient	Value for second-order ADRC
α_1	$-\frac{1}{8} \cdot (1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 + z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^3 + 1$
α_2	$z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^3$
β_0	$\frac{1}{b_0 T^2} \cdot \left[\frac{1}{4} \cdot (1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 - 2 \cdot \left(z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^3 + 2 z_{\mathrm{CL}} + 3 z_{\mathrm{ESO}} - 2\right)\right]$
β_1	$\frac{1}{b_0 T^2} \cdot \left[-(1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 + 2 \cdot (1+z_{\mathrm{CL}})^2 + 6 \cdot \left(z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^3 + 2 z_{\mathrm{CL}} z_{\mathrm{ESO}} + z_{\mathrm{ESO}}^2 + z_{\mathrm{ESO}} - 1\right)\right]$
β_2	$\frac{1}{b_0 T^2} \cdot \left[-\frac{1}{4} \cdot (1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 + 2 \cdot \left(-2 z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^3 + 3 z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^2 + 2 z_{\mathrm{CL}} z_{\mathrm{ESO}}^3 + 1\right)\right]$
γ_0	$\frac{(1-z_{\mathrm{CL}})^2}{b_0 T^2}$
γ_1	$\frac{-3 z_{\mathrm{ESO}} \cdot (1-z_{\mathrm{CL}})^2}{b_0 T^2}$
γ_2	$\frac{3 z_{\mathrm{ESO}}^2 \cdot (1-z_{\mathrm{CL}})^2}{b_0 T^2}$
γ_3	$\frac{-z_{\mathrm{ESO}}^3 \cdot (1-z_{\mathrm{CL}})^2}{b_0 T^2}$

Underlying system and tuning parameters are b_0 (plant gain), T (sampling interval), ω_{CL} (closed-loop bandwidth), k_{ESO} (observer bandwidth factor)

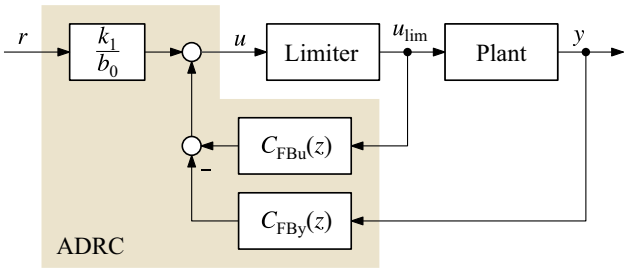


Fig. 7 Control loop with discrete-time ADRC, dual-feedback implementation using two transfer functions $C_{\mathrm{FBy}}(z)$ and $C_{\mathrm{FBu}}(z)$ given in (32) and (33)

$$C_{\mathrm{FBu}}(z) = -\frac{z^{-1}}{b_0} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \left(\boldsymbol{I} - z^{-1}\boldsymbol{A}_{\mathrm{ESO}}\right)^{-1} \cdot \boldsymbol{b}_{\mathrm{ESO}} = z^{-1} \cdot \frac{\sum_{i=0}^{n} \gamma_i z^{-i}}{1 + \sum_{i=1}^{n+1} \alpha_i z^{-i}}. \tag{33}$$

The transfer function coefficients are computed by inserting properly tuned controller gains $\boldsymbol{k}^{\mathrm{T}}$ and observer gains $\boldsymbol{l}$ in (31) and solving for α, β, and γ. Reference [18] gave tuning equations for these coefficients using bandwidth parameterization with the quasi-continuous controller design of Sect. 2.2. In this paper, we provide these coefficients for first- and second-order ADRC incorporating the discrete-time controller tuning of Sect. 4.1, now providing accurate results also for larger sampling intervals. They can be found in Tables 3 and 4. Note that the β coefficients are the same as in Sect. 5.1, i.e., the numerator polynomials of $C_{\mathrm{FBy}}(z)$ in (32) and $C_{\mathrm{FB}}(z)$ in (29) are identical.

Table 3 Discrete-time dual-feedback transfer function implementation: coefficients for first-order ADRC, obtained by discrete-time pole placement and bandwidth parameterization with $z_{\mathrm{CL}} = \mathrm{e}^{-\omega_{\mathrm{CL}} T}$ (closed loop) and $z_{\mathrm{ESO}} = \mathrm{e}^{-k_{\mathrm{ESO}} \omega_{\mathrm{CL}} T}$ (observer)

Coefficient	Value for first-order ADRC
α_1	$-2 z_{\mathrm{ESO}}$
α_2	z_{ESO}^2
β_0	$\frac{1}{b_0 T} \cdot \left[z_{\mathrm{CL}} z_{\mathrm{ESO}}^2 - 2 z_{\mathrm{ESO}} - z_{\mathrm{CL}} + 2\right]$
β_1	$\frac{1}{b_0 T} \cdot \left[2 z_{\mathrm{CL}} z_{\mathrm{ESO}} - 2 z_{\mathrm{CL}} z_{\mathrm{ESO}}^2 + z_{\mathrm{ESO}}^2 - 1\right]$
γ_0	$z_{\mathrm{CL}} z_{\mathrm{ESO}}^2 - 2 z_{\mathrm{ESO}} + 1$
γ_1	$z_{\mathrm{ESO}}^2 - z_{\mathrm{CL}} z_{\mathrm{ESO}}^2$
$\frac{k_1}{b_0}$	$\frac{1 - z_{\mathrm{CL}}}{b_0 T}$

A very efficient implementation was proposed in [18], exploiting the common denominator of $C_{\mathrm{FBy}}(z)$ and $C_{\mathrm{FBu}}(z)$ and choosing a transposed "direct form II" realization. This reduces the number of required storage variables during runtime to the minimum, and uses even less algebraic operations than the transfer function representation of Sect. 5.1—while keeping the flexibility and behavior of the state-space implementation regarding user-defined control signal limitation

Table 4 Discrete-time dual-feedback transfer function implementation: coefficients for second-order ADRC, obtained by discrete-time pole placement and bandwidth parameterization with $z_{\mathrm{CL}} = \mathrm{e}^{-\omega_{\mathrm{CL}} T}$ (closed loop) and $z_{\mathrm{ESO}} = \mathrm{e}^{-k_{\mathrm{ESO}} \omega_{\mathrm{CL}} T}$ (observer)

Coefficient	Value for second-order ADRC
α_1	$-3z_{\mathrm{ESO}}$
α_2	$3z_{\mathrm{ESO}}^2$
α_3	$-z_{\mathrm{ESO}}^3$
β_0	$\frac{1}{b_0 T^2} \cdot \left[\frac{1}{4} \cdot (1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 - 2 \cdot \left(z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^3 + 2z_{\mathrm{CL}} + 3z_{\mathrm{ESO}} - 2\right)\right]$
β_1	$\frac{1}{b_0 T^2} \cdot \left[-(1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 + 2 \cdot (1+z_{\mathrm{CL}})^2 + 6 \cdot \left(z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^3 + 2z_{\mathrm{CL}} z_{\mathrm{ESO}} + z_{\mathrm{ESO}}^2 + z_{\mathrm{ESO}} - 1\right)\right]$
β_2	$\frac{1}{b_0 T^2} \cdot \left[-\frac{1}{4} \cdot (1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 + 2 \cdot \left(-2z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^3 + 3z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^2 + 2z_{\mathrm{CL}} z_{\mathrm{ESO}}^3 + 1\right)\right]$
γ_0	$\frac{1}{8} \cdot (1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 - z_{\mathrm{ESO}} \cdot (z_{\mathrm{CL}}^2 z_{\mathrm{ESO}}^2 + 3)$
γ_1	$-\frac{1}{8} \cdot (1+z_{\mathrm{CL}})^2 \cdot (1+z_{\mathrm{ESO}})^3 + 3z_{\mathrm{ESO}}^2 + 1$
γ_2	$z_{\mathrm{ESO}}^3 \cdot (z_{\mathrm{CL}}^2 - 1)$
$\frac{k_1}{b_0}$	$\frac{(1-z_{\mathrm{CL}})^2}{b_0 T^2}$

and windup protection. For the first- and second-order case, the resulting controller structure is shown in Fig. 8a, b.

6 Error-based ADRC

Error-based ADRC is a variant presented in generalized form in [25]. It obtained its name from feeding the control error $e = r - y$ to the observer—instead of the plant output y in the "classical" case like the one shown in Sect. 2. To better differentiate the two approaches, we will accordingly denote the latter as "output-based" ADRC in this section.

The relation between output- and error-based ADRC was firmly established for the continuous-time case in [24]. Building on that, we want to extend the connections between output- and error-based ADRC to the discrete time domain in this article. The only discrete-time implementation of error-based ADRC so far in [25] was based on a bilinear transformation from its continuous-time transfer function representation. As we will see in this section, we can fully build on existing discrete-time ADRC structures instead. In doing so, we will derive new, equivalent variants of discrete-time error-based ADRC that match the three implementations covered in this paper: state-space, transfer function, and dual-feedback transfer function form. As an extra gain, this will extend error-based ADRC with the control signal limitation and anti-windup abilities of output-based ADRC.

6.1 Continuous-time error-based ADRC

For an in-depth introduction to error-based ADRC, we refer to [25]. A continuous-time block diagram of error-based ADRC is shown in Fig. 9. In contrast to [25], we explicitly are feeding the limited control signal $u_{\mathrm{lim}}(t)$ into to observer, to provide easy windup protection as in the ouput-based case in Sect. 2.1. The associated control law and observer dynamics are

$$u(t) = \frac{1}{b_0} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \hat{\boldsymbol{x}}(t), \tag{34}$$

$$\dot{\hat{\boldsymbol{x}}}(t) = \boldsymbol{A} \cdot \hat{\boldsymbol{x}}(t) - \boldsymbol{b} \cdot u_{\mathrm{lim}}(t) + \boldsymbol{l} \cdot \left(e(t) - \boldsymbol{c}^{\mathrm{T}} \cdot \hat{\boldsymbol{x}}(t)\right). \tag{35}$$

Not only the structure of these equations and the block diagram are similar to output-based ADRC: In the observer (35), $\boldsymbol{A}$, $\boldsymbol{b}$, and $\boldsymbol{c}^{\mathrm{T}}$ are exactly the same as in (2). This applies to the controller gains $\boldsymbol{k}^{\mathrm{T}}$ and observer gains $\boldsymbol{l}$, as well, which means that all existing tuning methods can be used here without modification.

6.2 Discrete-time state-space implementation

To derive the discrete-time state-space version of error-based ADRC, we can largely build on the results from output-based ADRC in Sect. 3.1. The control law (34) is a static state feedback, and hence trivial to translate to the discrete time domain:

$$u(k) = \frac{1}{b_0} \cdot \left(\boldsymbol{k}^{\mathrm{T}}\ 1\right) \cdot \hat{\boldsymbol{x}}(k). \tag{36}$$

We only have to derive a discretized observer from (35). By visual inspection of Figs. 1 and 9, only two differences between the observers can be found:

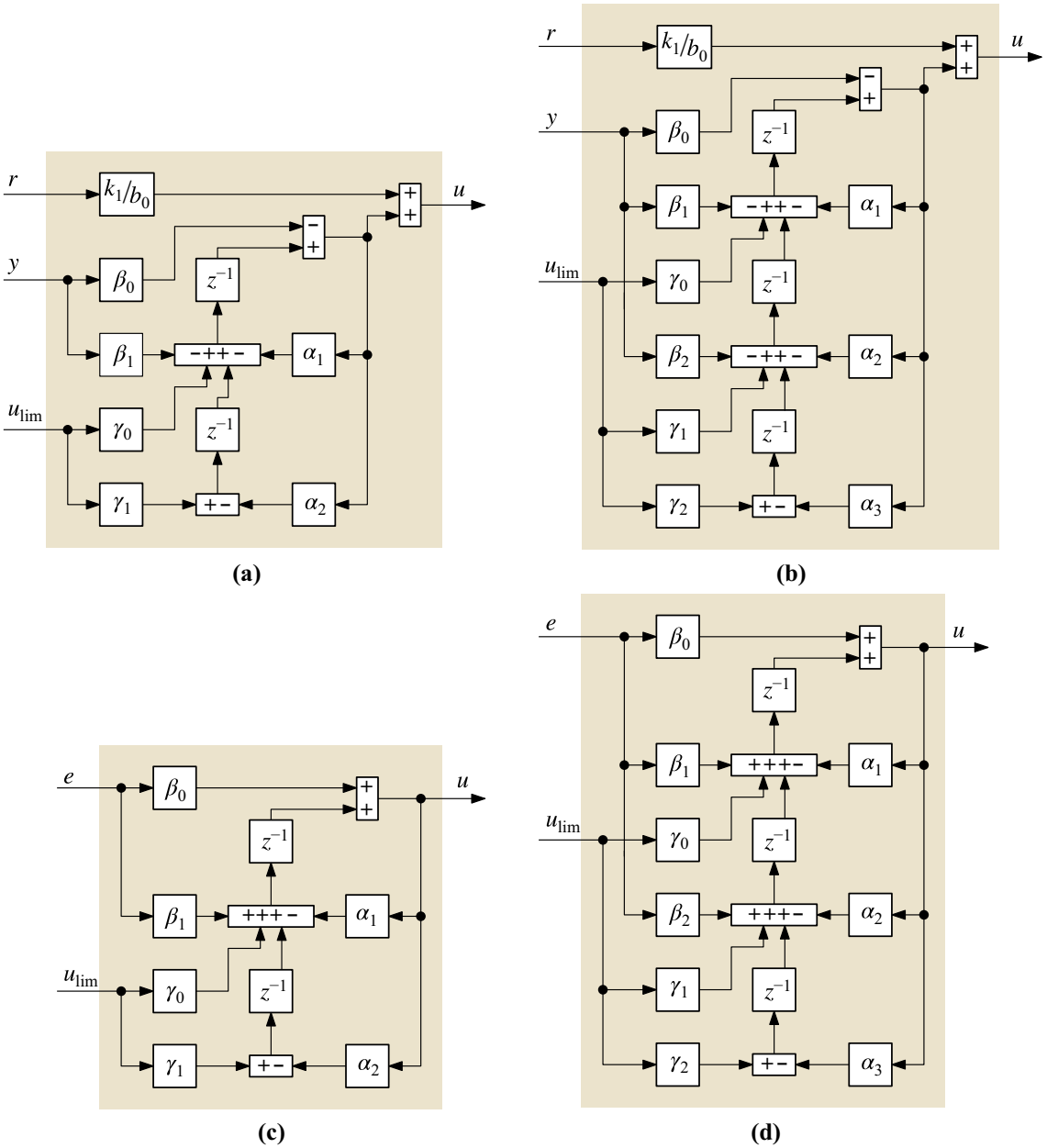


Fig. 8 Dual-feedback transfer function implementations of discrete-time ADRC variants with minimum footprint. **a** Output-based, first-order. **b** Error-based, second-order. **c** Error-based, second-order. **d** Error-based, second-order

– For error-based ADRC, the input $e(t)$ is being used instead of $y(t)$, and
– the limited controller output $u_{\lim}(t)$ enters the observer with a negative sign.

This enables us to take a very convenient shortcut. We can simply reuse the discrete-time observer Eq. (9) from Sect. 3.1, exchange $y(k)$ for $e(k)$, and change the sign for $u_{\lim}(k)$. The properly discretized observer for error-based ADRC now reads:

$$\hat{\boldsymbol{x}}(k) = \boldsymbol{A}_{\text{ESO}} \cdot \hat{\boldsymbol{x}}(k-1) - \boldsymbol{b}_{\text{ESO}} \cdot u_{\lim}(k-1) + \boldsymbol{l} \cdot e(k) \tag{37}$$

with $\boldsymbol{A}_{\text{ESO}}$ and $\boldsymbol{b}_{\text{ESO}}$ as well as controller/observer gains being the same as in the output-based case. The block diagram shown in Fig. 10 is, consequently, also very similar to Fig. 2.

Please remember that even though this derivation could be presented in an extremely condensed form, it is the first discrete-time implementation of error-based ADRC in state space. Owing to the equivalence to output-based ADRC at the

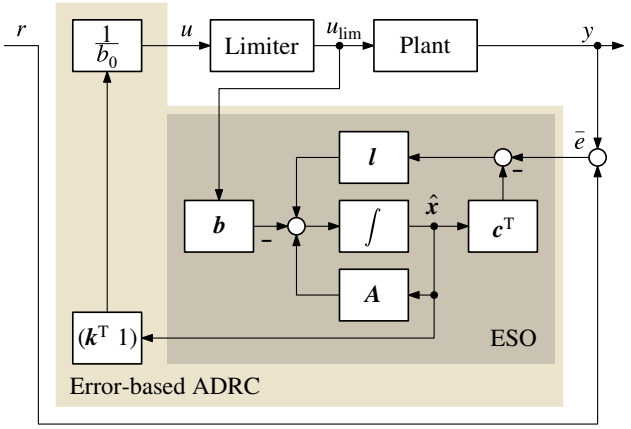


Fig. 9 Structure of a control loop with error-based ADRC (continuous-time state-space version) and a user-defined control signal limiter

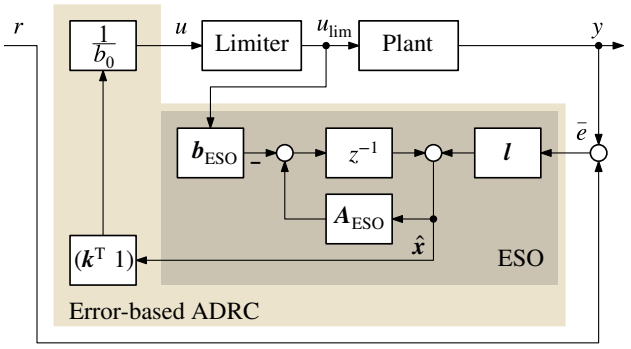


Fig. 10 Control loop with discrete-time error-based ADRC, state space implementation

core observer dynamics, all findings of [8] regarding the best discretization strategy for linear ADRC automatically apply here as well (zero-order hold discretization in conjunction with the current observer approach).

This also means that there is no need for a "custom" discretization of error-based ADRC at transfer function level, as done in [25]—we can instead follow the same route of exact transfer function representations as presented in Sect. 5 of this article, and will do so right next.

6.3 Discrete-time transfer function implementation

As the counterpart of the output-based version in Sect. 5.1, a transfer function implementation of discrete-time error-based ADRC can now easily be derived by following the same steps. We once again start with the z-domain representation of control law (36) and observer (37), and use $u(z) = u_{\text{lim}}(z)$ as an observer input, as we will lose the dedicated feedback path of the limited controller output:

$$u(z) = \frac{1}{b_0} \cdot \left(\boldsymbol{k}^{\text{T}}\ 1\right) \cdot \hat{\boldsymbol{x}}(z), \tag{38}$$

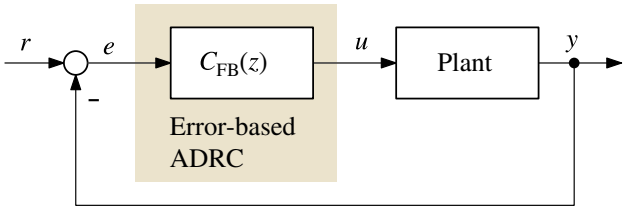


Fig. 11 Control loop with discrete-time error-based ADRC, transfer function implementation

$$\hat{\boldsymbol{x}}(z) = z^{-1} \cdot \boldsymbol{A}_{\text{ESO}} \cdot \hat{\boldsymbol{x}}(z) - z^{-1} \cdot \boldsymbol{b}_{\text{ESO}} \cdot u(z) + \boldsymbol{l} \cdot e(z). \tag{39}$$

The closed-loop observer equation is obtained by putting (38) into (39):

$$\hat{\boldsymbol{x}}(z) = \boldsymbol{\Phi}_{\text{ESO}} \cdot \boldsymbol{l} \cdot e(z),$$

where $\boldsymbol{\Phi}_{\text{ESO}}$ is identical to (24) in the output-based case. Inserting this equation back into (38) yields the control law for the transfer function implementation of error-based ADRC, which is also shown in Fig. 11:

$$u(z) = \underbrace{\frac{1}{b_0} \cdot \left(\boldsymbol{k}^{\text{T}}\ 1\right) \cdot \boldsymbol{\Phi}_{\text{ESO}} \cdot \boldsymbol{l}}_{C_{\text{FB}}(z)} \cdot e(z). \tag{40}$$

Equivalent to the continuous-time derivation in [24], only one transfer function is required here. And, above all, exactly the same transfer function $C_{\text{FB}}(z)$ given in (29) is being used for output- and error-based ADRC, with the same α and β coefficients from Tables 1 and 2.

Comparing control laws and block diagrams of discrete-time output- and error-based ADRC in Figs. 6 and 11, it is clear that output-based ADRC can be seen as an extension of error-based ADRC, with an additional reference signal prefilter $C_{\text{PF}}(z)$—as in the continuous-time case [24].

In output-based ADRC, the dynamics of reference tracking and disturbance rejection cannot be independently chosen, the prefilter is tuned for a critically damped closed-loop response. Being a more "bare-bones" version, error-based ADRC therefore allows to be more flexibly combined with an independently tuned prefilter for an actual two-degrees-of-freedom design.

6.4 Discrete-time dual-feedback transfer function implementation

With the discrete-time state-space implementation of error-based ADRC in Sect. 6.2 at hand, a dual-feedback transfer function variant can easily be derived, as well. The z-transform of the observer (37),

$$\hat{\boldsymbol{x}}(z) = z^{-1} \cdot \boldsymbol{A}_{\text{ESO}} \cdot \hat{\boldsymbol{x}}(z) - z^{-1} \cdot \boldsymbol{b}_{\text{ESO}} \cdot u_{\text{lim}}(z) + \boldsymbol{l} \cdot e(z)$$

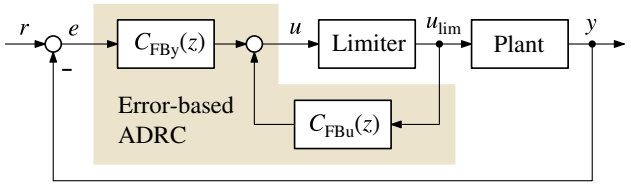


Fig. 12 Control loop with discrete-time error-based ADRC, dual-feedback transfer function implementation

is being solved for $\hat{\boldsymbol{x}}(z)$, yielding:

$$\hat{\boldsymbol{x}}(z) = \left(\boldsymbol{I} - z^{-1}\boldsymbol{A}_{\mathrm{ESO}}\right)^{-1} \cdot \left(-z^{-1} \cdot \boldsymbol{b}_{\mathrm{ESO}} \cdot u_{\mathrm{lim}}(z) + \boldsymbol{l} \cdot e(z)\right),$$

which can be inserted into the control law (38):

$$u(z) = \frac{1}{b_0} \cdot \begin{pmatrix}\boldsymbol{k}^{\mathrm{T}} & 1\end{pmatrix} \cdot \left(\boldsymbol{I} - z^{-1}\boldsymbol{A}_{\mathrm{ESO}}\right)^{-1} \cdot \left(-z^{-1} \cdot \boldsymbol{b}_{\mathrm{ESO}} \cdot u_{\mathrm{lim}}(z) + \boldsymbol{l} \cdot e(z)\right). \tag{41}$$

By inspection and comparison with the output-based counterpart (31), we can make the important observation that (41) can be represented using the two already introduced transfer functions $C_{\mathrm{FBy}}(z)$ and $C_{\mathrm{FBu}}(z)$:

$$u(z) = C_{\mathrm{FBy}}(z) \cdot e(z) + C_{\mathrm{FBu}}(z) \cdot u_{\mathrm{lim}}(z), \tag{42}$$

where $C_{\mathrm{FBy}}(z)$ and $C_{\mathrm{FBu}}(z)$ are identical to their definition in (32) and (33). The required α, β, γ coefficients can, consequently, be reused in unmodified form as given in Tables 3 and 4 for the first- and second-order case.

Please note that we kept the subscript "y" as part of $C_{\mathrm{FBy}}(z)$ in the control law (42) and its visualization in Fig. 12 to emphasize that exactly the same transfer functions from the output-based variant are being reused here, even though $C_{\mathrm{FBy}}(z)$ is now being fed with the control error e—which, in turn, of course also contains y.

As a final step, a very efficient implementation of this control law can be found as well using the ideas from [18]. With the common denominator of $C_{\mathrm{FBu}}(z)$ and $C_{\mathrm{FBy}}(z)$, the control law (42) can be written as

$$u(z) = \frac{z^{-1} \cdot \left(\sum_{i=0}^{n} \beta_i z^{-i}\right) \cdot u_{\mathrm{lim}}(z) + \left(\sum_{i=0}^{n} \gamma_i z^{-i}\right) \cdot e(z)}{1 + \sum_{i=1}^{n+1} \alpha_i z^{-i}}. \tag{43}$$

A direct form II implementation of (43) reduces both storage variables and algebraic operations required during runtime to a minimum. The resulting implementations are shown in Fig. 8c, d for the first- and second-order case, allowing a direct comparison with their output-based counterparts.

6.5 Summary

We have now introduced three new discrete-time implementation of error-based ADRC—so when to use what? Advantages and disadvantages of each are the same as for their output-based counterparts. Main aspects are: familiarity with the structure, possible reuse of software building blocks, e. g. for transfer functions, control signal limitation and windup protection, and the computational footprint. In Table 5, we summarize relevant characteristics as a decision support.

7 Example

In this section, a power electronics application (DC–DC converter) to be controlled by first-order output- and error-based ADRC is being studied. [1] As we developed error-based ADRC implementations in this paper that exhibit exactly the same dynamic disturbance rejection behavior as their output-based counterparts, the equivalence between output- and error-based ADRC in this regard should therefore be visible. Furthermore, the anti-windup abilities shall be demonstrated that error-based ADRC gained in the implementations introduced in this paper by keeping the separate feedback path for the limited controller output in both the state-space and the dual-feedback variant. For comparability down to the level of individual sampling intervals, we will perform this comparison in a simulation environment.

An overview of the DC–DC converter and its control structure with error-based ADRC is shown in Fig. 13. The converter parameters are $V_{\mathrm{Input}} = 24\,\mathrm{V}$, $L = 33\,\mu\mathrm{H}$, $C = 100\,\mu\mathrm{F}$, base load $R = 100\,\Omega$, $f_{\mathrm{Switch}} = 50\,\mathrm{kHz} = \frac{1}{T}$, peak-current mode slope compensation of 2 A per cycle. To limit not only the absolute values, but also the slope of the converter current during transients, the voltage controller is equipped with a magnitude and rate limitation block configured for peak current commands from the interval [0 A, 6 A] with a rate limit of $\pm 20\,\mathrm{A/ms}$. To dynamically decouple reference tracking from disturbance rejection, a first-order low-pass setpoint filter is added, with a time constant of 750 µs for a 98 % settling time of 3 ms. ADRC tuning is performed with a bandwidth $\omega_{\mathrm{CL}} = 4000\,\mathrm{rad/s}$ for an 98 % settling time of 1 ms, and a relative observer bandwidth $k_{\mathrm{ESO}} = 5$. As shown in [10], plant modeling for ADRC is extremely easy in this application, depending only on the output capacitor with $b_0 = \frac{1}{C} = 10000\,\mathrm{V/As}$. The sampling frequency of the discrete-time implementation corresponds

[1] For output-based ADRC, this is a replication of the example in [18].

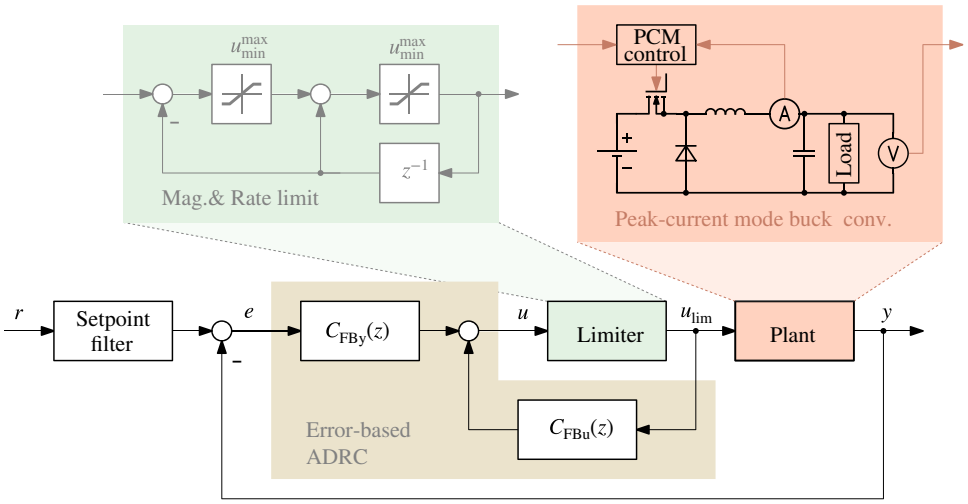


Fig. 13 Schematic of a voltage control loop around a peak-current mode controlled step-down DC–DC converter. Discrete-time error-based ADRC in the dual-feedback transfer function implementation is being used as voltage controller, enhanced by a first-order low-pass filter to adjust the setpoint tracking dynamics. To limit the absolute value as well as the slope of the converter current during transients, the voltage controller is equipped with a rate and magnitude limitation block

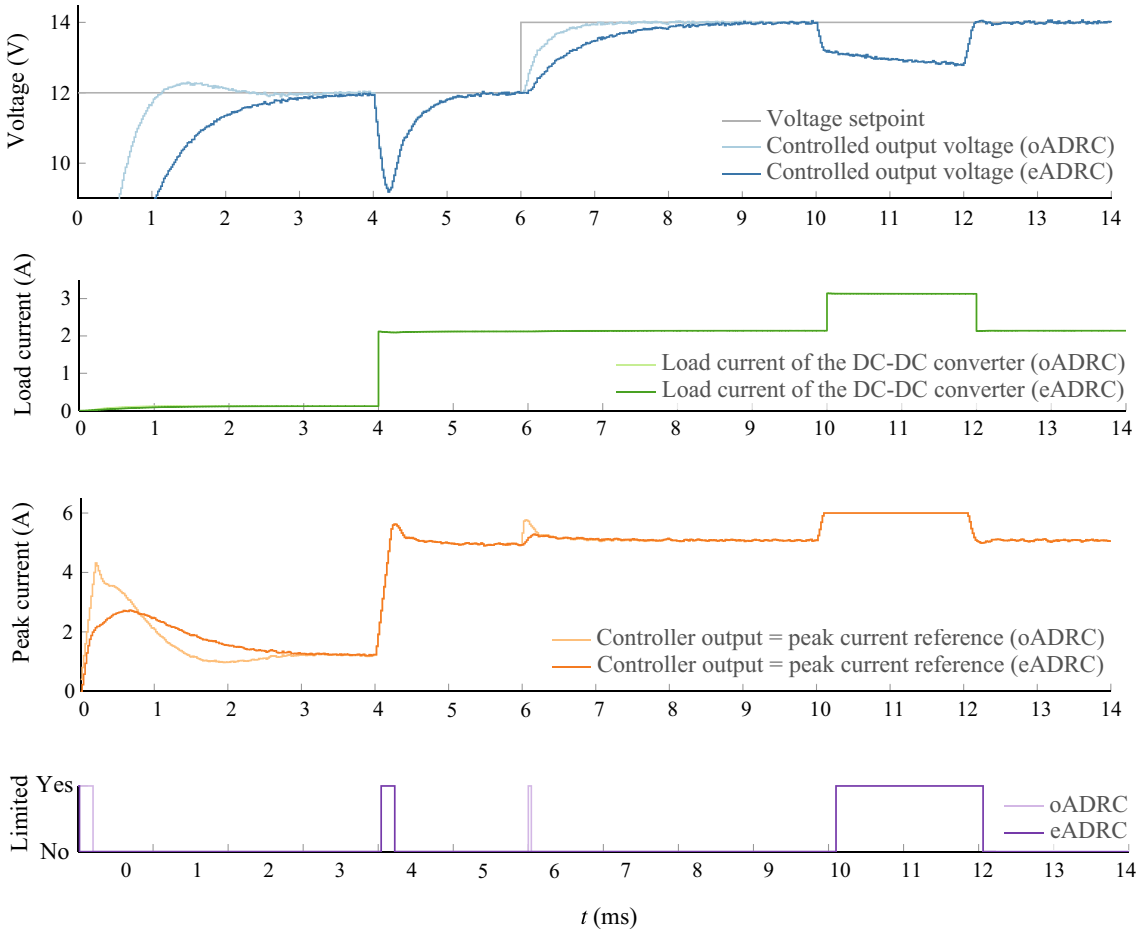


Fig. 14 Simulation results for the application example from Fig. 13. In each diagram, the result of output-based ADRC (oADRC) is plotted with brighter, and error-based ADRC (eADRC) with darker color. On can very well spot the identical behavior of oADRC and eADRC during load transients and when returning from saturation—irrespective of whether limited by magnitude or by rate (active limitation is shown in the bottom diagram). As expected, only the reference tracking behavior differs. The eADRC variant was equipped with an additional setpoint filter for a slower, more smooth response in a true two-degrees-of-freedom design. Note that the voltage drop between 10 ms $< t <$ 12 ms does not result from the controllers inability to reject disturbances, but from the load current being (deliberately chosen) so high that the input current the (saturated) controller is allowed to supply does not suffice to prevent the capacitor from discharging

Table 5 Overview of discrete-time implementations for output-based and error-based ADRC presented in this article

Implementation	Output-based	Error-based	Characteristics
State space	Sect. 3.1	Sect. 6.2	▲Discrete-time equivalent of the original continuous-time structure of linear ADRC. ▲Flexible control signal limitation and windup protection. Runtime: ▲Minimum number of storage variables, ▼maximum number of coefficients and operations.
Transfer function	Sect. 5.1	Sect. 6.3	▲Strong similarity to classical digital control using feedback controller and prefilter. ▼Control signal limitation only via clamped integrator. Runtime: ▼Maximum number of storage variables, ▲reduced number of coefficients and operations.
Dual-feedback TF	Sect. 5.2	Sect. 6.4	▲Full replication of the state-space behavior using transfer functions, ▼but with uncommon structure. ▲Flexible control signal limitation and windup protection. Runtime: ▲Minimum number of storage variables, coefficients, and operations.

To help decide for a variant, a (coarse) binary assessment of the characteristics is given, hinting ▲advantages and ▼disadvantages

to the switching frequency. To be closer to practice, measurement noise with $\sigma = 20\,\text{mV}$ is added to the output voltage, and a full sampling interval assumed as the delay between measurement and updated controller output.

The scenario consists of startup at (almost) no load, a jump to full load at $t = 4\,\text{ms}$, a setpoint change at $t = 6\,\text{ms}$, and a temporary overload condition between $t = 10\,\text{ms}$ and $t = 12\,\text{ms}$—the latter to deliberately drive the controller into saturation. The simulation is performed twice: once with error-based ADRC (including the setpoint filter), and once with output-based ADRC (without setpoint filter). Results are presented in Fig. 14.

The simulation results confirm that the discrete-time error-based implementations introduced in this article deliver identical disturbance rejection behavior as their output-based counterparts, down to the scope of individual samples. As an additional plus, they can now benefit from the same windup protection options.

8 Conclusion

Practical implementations of ADRC are usually carried out in software, i.e., in a discrete time domain. This article provided both a summary and an extension of discrete-time implementations of linear ADRC. As a first contribution, controller tuning was performed using bandwidth parameterization with discrete-time pole placement, departing from the prevalent quasi-continuous design procedure. A bridge to the existing tuning rules could be built, while also demonstrating the necessity of accurate results for low sampling frequencies. For two existing discrete-time transfer function implementations of ADRC, coefficients were rederived based on accurate discrete-time pole placement.

A second bridge was built from "classical" (output-based) to error-based ADRC in the discrete time domain. It was shown that existing implementations need only slight restructuring, and transfer functions, coefficients, tuning rules can all be reused for error-based ADRC. In one sweep, three new error-based ADRC implementations could be derived in this article, which are all direct counterparts of existing output-based variants. Table 5 summarizes all variants presented here along with some main characteristics, to assist with making a choice.

Error- and output-based implementations now exhibit exactly the same inner-loop behavior (disturbance rejection), rendering custom discrete-time implementations of error-based ADRC—as done e. g. in [25]—unnecessary. As an important feature for real-world applications, error- and output-based ADRC can now be equipped with the same control signal limitation and windup protection capabilities.

Be it the necessity of a true two-degrees-of-freedom design, windup protection, a low footprint during runtime, or structural similarity to solutions of classical digital control—the discrete-time ADRC variants introduced and discussed in

this article can now cater a larger variety of practical needs than ever. What a great time to put ADRC into practice!

Declarations

Conflict of interest The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Han, J. (2009). From PID to active disturbance rejection control. *IEEE Transactions on Industrial Electronics, 56*(3), 900–906. https://doi.org/10.1109/TIE.2008.2011621
2. Zheng, Q., & Gao, Z. (2010). On practical applications of active disturbance rejection control. *Proceedings of the 29th Chinese Control Conference* (pp. 6095–6100).
3. Zheng, Q., & Gao, Z. (2018). Active disturbance rejection control: Some recent experimental and industrial case studies. *Control Theory and Technology, 16*(4), 301–313. https://doi.org/10.1007/s11768-018-8142-x
4. Talole, S. E. (2018). Active disturbance rejection control: Applications in aerospace. *Control Theory and Technology, 16*, 314–323. https://doi.org/10.1007/s11768-018-8114-1
5. Fareh, R., Khadraoui, S., Abdallah, M. Y., Baziyad, M., & Bettayeb, M. (2021). Active disturbance rejection control for robotic systems: A review. *Mechatronics, 80*, 102671. https://doi.org/10.1016/j.mechatronics.2021.102671
6. Gao, Z. (2006). Active disturbance rejection control: A paradigm shift in feedback control system design. *Proceedings of the 2006 American Control Conference* (pp. 2399–2405). https://doi.org/10.1109/ACC.2006.1656579
7. Gao, Z. (2003). Scaling and bandwidth-parameterization based controller tuning. *Proceedings of the 2003 American Control Conference* (pp. 4989–4996). https://doi.org/10.1109/ACC.2003.1242516
8. Miklosovic, R., Radke, A., & Gao, Z. (2006). Discrete implementation and generalization of the extended state observer. *Proceedings of the 2006 American Control Conference* (pp. 2209–2214). https://doi.org/10.1109/ACC.2006.1656547
9. Madonski, R., Gao, Z., & Łakomy, K. (2015). Towards a turnkey solution of industrial control under the active disturbance rejection paradigm. *54th Annual Conference of the Society of Instrument and Control Engineers of Japan (SICE)* (pp. 616–621). https://doi.org/10.1109/SICE.2015.7285478
10. Herbst, G. (2016). Practical active disturbance rejection control: Bumpless transfer, rate limitation, and incremental algorithm. *IEEE Transactions on Industrial Electronics, 63*(3), 1754–1762. https://doi.org/10.1109/TIE.2015.2499168
11. Madonski, R., & Herman, P. (2015). Survey on methods of increasing the efficiency of extended state disturbance observers. *ISA Transactions, 56*, 18–27. https://doi.org/10.1016/j.isatra.2014.11.008
12. Fu, C., & Tan, W. (2016). Tuning of linear ADRC with known plant information. *ISA Transactions, 65*, 384–393. https://doi.org/10.1016/j.isatra.2016.06.016
13. Zhou, R., & Tan, W. (2019). Analysis and tuning of general linear active disturbance rejection controllers. *IEEE Transactions on Industrial Electronics, 66*(7), 5497–5507. https://doi.org/10.1109/TIE.2018.2869349
14. Tian, G., & Gao, Z. (2007). Frequency response analysis of active disturbance rejection based control system. *IEEE International Conference on Control Applications* (pp.1595–1599). https://doi.org/10.1109/CCA.2007.4389465
15. Huang, C., & Gao, Z. (2013). On transfer function representation and frequency response of linear active disturbance rejection control. *Proceedings of the 32th Chinese Control Conference* (pp. 72–77)
16. Zheng, Q., & Gao, Z. (2016). Active disturbance rejection control: Between the formulation in time and the understanding in frequency. *Control Theory and Technology, 14*(3), 250–259. https://doi.org/10.1007/s11768-016-6059-9
17. Herbst, G. (2021). Transfer function analysis and implementation of active disturbance rejection control. *Control Theory and Technology, 19*, 19–34. https://doi.org/10.1007/s11768-021-00031-5
18. Herbst, G. (2021). A minimum-footprint implementation of discrete-time ADRC. *2021 European Control Conference (ECC)* (pp. 107–112). https://doi.org/10.23919/ECC54610.2021.9655120
19. Michałek, M.M. (2016). Robust trajectory following without availability of the reference time-derivatives in the control scheme with active disturbance rejection. *Proceedings of the 2016 American Control Conference* (pp. 1536–1541). https://doi.org/10.1109/ACC.2016.7525134
20. Mandali, A., Dong, L., & Morinec, A. (2020). Robust controller design for automatic voltage regulation. *Proceedings of the 2020 American Control Conference* (pp. 2617–2622). https://doi.org/10.23919/ACC45564.2020.9147208
21. Lechekhab, T. E., Manojlović, S. M., Stanković, M. R., Madonski, R., & Simić, S. M. (2021). Robust error-based active disturbance rejection control of a quadrotor. *Aircraft Engineering and Aerospace Technology, 93*(1), 89–104. https://doi.org/10.1108/AEAT-12-2019-0266
22. Madonski, R., Łakomy, K., & Yang, J. (2021). Simplifying ADRC design with error-based framework: Case study of a DC-DC buck power converter. *Control Theory and Technology, 19*, 94–112. https://doi.org/10.1007/s11768-021-00035-1
23. Huang, T., Hu, G., Yan, Y., Zeng, D., & Meng, Z. (2022). Combined feedforward and error-based active disturbance rejection control for diesel particulate filter thermal regeneration. *ISA Transactions*. https://doi.org/10.1016/j.isatra.2022.09.013
24. Madonski, R., Herbst, G., & Stankovic, M. (2023). ADRC in output and error form: Connection, equivalence, performance. *Control Theory and Technology*. https://doi.org/10.1007/s11768-023-00129-y
25. Madonski, R., Shao, S., Zhang, H., Gao, Z., Yang, J., & Li, S. (2019). General error-based active disturbance rejection control for swift industrial implementations. *Control Engineering Practice, 84*, 218–229. https://doi.org/10.1016/j.conengprac.2018.11.021
26. Åström, K. J., & Murray, R. M. (2021). *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton, NJ, USA: Princeton University Press.
27. Herbst, G. (2013). A simulative study on active disturbance rejection control (ADRC) as a control tool for practitioners. *Electronics, 2*(3), 246–279. https://doi.org/10.3390/electronics2030246
28. Franklin, G. F., Workman, M. L., & Powell, D. (1997). *Digital Control of Dynamic Systems*. Boston, MA, USA: Addison-Wesley Longman Publishing.
29. Miklosovic, R., & Radke, A. (2007). High performance tracking control for the practitioner. *Proceedings of the 2007 American Control Conference*, pp. 3009–3014. https://doi.org/10.1109/ACC.2007.4283051
30. Zhang, Y., Zhang, Y., Wang, J., & Ma, R. (2013). An active disturbance rejection control of induction motor using DSP+FPGA. *25th Chinese Control and Decision Conference (CCDC)* (pp. 4047–4052). https://doi.org/10.1109/CCDC.2013.6561659
31. Stanković, M. R., Manojlović, S. M., Simić, S. M., Mitrović, S. T., & Naumović, M. B. (2016). FPGA system-level based design of

multi-axis ADRC controller. *Mechatronics, 40*, 146–155. https://doi.org/10.1016/j.mechatronics.2016.10.005
32. Ahi, B., & Nobakhti, A. (2018). Hardware implementation of an ADRC controller on a gimbal mechanism. *IEEE Transactions on Control Systems Technology, 26*(6), 2268–2275. https://doi.org/10.1109/TCST.2017.2746059
33. Desai, R., Patre, B.M., & Pawar, S.N. (2018). Active disturbance rejection control with adaptive rate limitation for process control application. *2018 Indian Control Conference (ICC)* (pp. 131–136). https://doi.org/10.1109/INDIANCC.2018.8307966
34. Ramírez-Neria, M., Luviano-Juárez, A., Lozada-Castillo, N., Ochoa-Ortega, G., & Madonski, R. (2020). Discrete-time active disturbance rejection control: A delta operator approach. In A. Bartoszewicz, J. Kabziński, J. Kacprzyk (Eds.), *Advanced, Contemporary Control* (pp. 1383–1395). Cham, Switzerland: Springer International Publishing. https://doi.org/10.1007/978-3-030-50936-1_115
35. Peng, Y., Vrančič, D., & Hanus, R. (1996). Anti-windup, bumpless, and conditioned transfer techniques for PID controllers. *IEEE Control Systems Magazine, 16*(4), 48–57. https://doi.org/10.1109/37.526915
36. Åström, K.J., & Hägglund, T. (2006). *Advanced PID Control*. Durham, NC, USA: ISA—The Instrumentation, Systems, and Automation Society.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Gernot Herbst received Dipl.-Ing. and Dr.-Ing. degrees in Electrical Engineering from Chemnitz University of Technology, Germany, in 2006 and 2011, respectively. He was a Scientific Assistant at Chemnitz University of Technology from 2006 to 2012, before joining Siemens AG, Chemnitz, Germany, as an R&D Engineer. Since 2021, he is with the University of Applied Sciences Zwickau, Germany. His research interests revolve around bringing modern control into industrial practice, especially using active disturbance rejection control, as well as modeling and control of power electronics systems.



Rafal Madonski received M.Eng. and Ph.D. degrees in Automation and Robotics from Poznan University of Technology, Poland, in 2010 and 2016, respectively. He was a Postdoctoral Researcher in the School of Automation, Southeast University, China, between 2017 and 2019. He was a Visiting Researcher at Cleveland State University, USA, and the Chinese Academy of Sciences, Beijing, China. Since 2020, he is an Associate Professor in the Energy and Electricity Research Center, Jinan University, Zhuhai, China. He is a Senior Member of IEEE. His research interests include active disturbance rejection control and disturbance observers.